Aegis: Consistent Data Management for Agentic Transactions with Adaptive Concurrency Control in Data-Agent Systems

Abstract

LLM-based data agents provide a new interface for executing data-intensive tasks over enterprise data. They dynamically generate multi-step workflows that involve reasoning, data access, analysis, and updates. Such tasks require task-wide data consistency because early reads can affect later reasoning and update decisions. Stale or inconsistent data may therefore lead to incorrect decisions, constraint violations, or irreversible side effects. However, existing agent execution frameworks mainly focus on workflow safety, observability, and recoverability, while ignoring data consistency.

This paper introduces `Aegis`, a transactional concurrency control infrastructure designed specifically for LLM execution environments. `Aegis` abstracts the data operations performed by a data agent for a user task as an ***agentic transaction*** and provides transaction management for it. To support the dynamically generated, long-running, phase-structured, and costly-to-abort nature of agentic transactions, `Aegis` introduces **ATCC**, a phase-aware adaptive concurrency control algorithm. ATCC uses reinforcement learning to balance optimistic execution and targeted locking, preserving optimistic execution in low-risk stages while selectively locking high-risk data objects. `Aegis` further employs priority-based lock scheduling to reduce contention-induced tail latency and wasted reasoning cost. Experiments show that `Aegis` improves throughput by up to four orders of magnitude and reduces tail latency by up to 90% compared with state-of-the-art concurrency control schemes.

Keywords

Data agent; data consistency; transaction processing; concurrency control;

ACM Reference Format:

. 2027. Aegis: Consistent Data Management for Agentic Transactions with Adaptive Concurrency Control in Data-Agent Systems. In *Proceedings of SIGMOD (Conference acronym 'XX)*. ACM, New York, NY, USA, 16 pages. https://doi.org/XXXXXXX.XXXXXXX

1 Introduction

LLM-based AI agents [16, 52, 60, 72, 73, 89, 91] provide a new way for users to execute data-intensive tasks over increasingly large and complex enterprise data environments. These tasks often go beyond simple queries and require data retrieval, transformation, and analysis over managed data. Data agents allow users to express goals in natural language, while automatically interpreting these intents, constructing execution workflow, and issuing data operations over managed data objects [42, 63, 67, 68, 84, 94]. This shifts query formulation and workflow coordination from users to agents, reducing user effort. As a result, data agents are gaining increasing attention in both academia and industry, and are being explored in critical business workflows such as digital payments, e-commerce, financial analysis, and service reservation [5, 7, 54, 81].

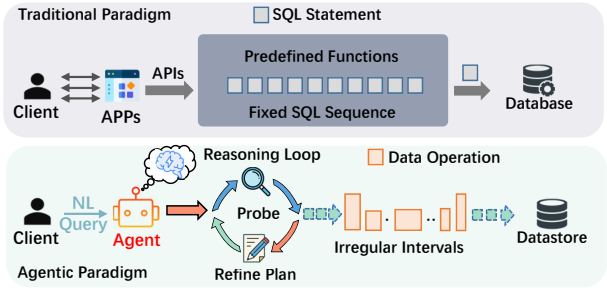

Figure 1: Comparison of traditional predefined transaction processing (TP) and agent-driven task processing with dynamic generation.

Data agents use the *agent environment* to ground reasoning in external state and execute actions beyond text generation. This environment provides shared and mutable resources for task processing, such as databases, file systems, in-memory stores, and system metadata [40]. As agents interact with these shared resources, maintaining a consistent task-wide data view becomes critical for reliable execution. A data-agent task may span multiple rounds of reasoning, decisions, and execution. Since later steps depend on earlier data results, stale or inconsistent reads may mislead reasoning and planning, causing incorrect decisions, constraint violations, or irreversible external side effects.

An example. Consider a digital banking scenario where an agent adjusts the credit limit of a corporate customer. The agent first reads historical records to build the customer's profile. It then retrieves the latest financial status, such as recent payment behavior and market conditions. Finally, it updates the credit limit. If another concurrent agent updates the customer's repayment status between the initial read and the final credit-limit update, the first agent may make an incorrect credit adjustment over a stale view.

Existing studies on agent execution mainly improve workflow-level controllability, observability, and recoverability. Agent execution frameworks and harnesses provide the runtime structure for coordinating agent planning, tool invocation, state management, and supervision. They make long-running or multi-agent workflows easier to constrain, inspect, resume, and debug, reducing failures caused by unconstrained tool use or opaque execution histories [1, 10, 35, 43, 47, 53, 80, 85]. Recent transaction-like agent systems push this direction further by introducing task boundaries and recovery-oriented mechanisms, such as effect tracking,

Conference acronym 'XX, Woodstock, NY

ACM ISBN 978-1-4503-XXXX-X/2027/06
https://doi.org/XXXXXXX.XXXXXXX

validation, compensation, or rollback, so that partially executed agent workflows can be detected, repaired, or reverted after failures [18, 44, 54, 79]. However, these mechanisms mainly focus on workflow protection. They do not guarantee a consistent data view across the data operations, nor do they enforce isolation among concurrent data-agent tasks. This leaves online concurrency control unresolved when agents concurrently access shared data objects.

To address this, we introduce transaction semantics for consistent data operations into data-agent execution through ***agentic transactions***. An agentic transaction encapsulates the data operations performed during an agent task, so that the agent observes a consistent view during execution and its updates become visible atomically only if the task commits. Building on this, we propose **Aegis**, a transactional concurrency control infrastructure for data-agent systems.

Supporting agentic transactions for agents introduces concurrency-control challenges that are not well addressed by traditional database mechanisms. **First**, long and irregular reasoning times make fixed concurrency-control protocols ineffective. Traditional OLTP transactions are typically short-lived, predefined, and have stable access patterns, as shown in Figure 1. In contrast, agentic transactions interleave data operations with multi-round reasoning. The intervals are irregular and hard to predict. Pessimistic Concurrency Control (PCC) [14, 51, 61, 88] prevents conflicts via locking but will lead to long blocking and inflated tail latency. Optimistic Concurrency Control (OCC) [24, 45, 74, 86, 87] avoids blocking, but will incur high abort rates due to frequent conflicts and stale reads.

Second, evolving access patterns make static hybrid protocols ineffective. Agentic transactions decide and revise their data operations based on intermediate results and runtime feedback, so their access sets are not fully known at the beginning and may change during execution. Existing hybrid schemes [17, 19, 71, 76, 77, 96] combine PCC and OCC, but they usually rely on conventional stable workload-level statistics to make static protocol choices. Such assumptions do not fit agentic transactions, as the evolving execution can shift contention windows over time, making fixed hybrid decisions poorly timed.

Third, abort costs are high and heterogeneous. Aborting an agentic transaction discards more than executed operations. It also invalidates reasoning state, intermediate artifacts, and plan revisions. Even with proper lock timing, transactions may still compete for the same data items. Uniform lock scheduling can block near-complete or high-cost transactions behind newly started ones. It can also force expensive re-execution and waste resources.

To address these challenges, Aegis first introduces an adaptive concurrency control algorithm, **ATCC**, that dynamically applies optimistic or pessimistic execution according to runtime contention and transaction behavior. Instead of committing to a fixed protocol at transaction start, ATCC preserves optimistic execution when conflict risk is low and switches selected operations to pessimistic protection when conflicts become likely.

Second, Aegis uses reinforcement learning (RL) to learn a phase-aware locking policy from runtime feedback, guiding the adaptive decisions in ATCC. Although agent execution flows vary, many data-agent task execution flows exhibit a coarse phase structure, moving from broad exploratory reads to refined access and then to a small number of critical updates [50, 79] (Section 2.2). Based on this observation, the trained policy can capture the delayed trade-off between immediate blocking cost and future abort cost. It allows ATCC to preserve optimistic execution in low-risk stages and selectively locks high-risk data objects, reducing aborts without excessive blocking.

Third, Aegis integrates priority-based locking to schedule lock requests under contention. Specifically, Aegis derives transaction priority from runtime factors such as execution progress, execution time, and estimated reasoning cost. The priority is then used to order conflicting lock requests, favoring transactions whose abort or prolonged waiting would waste more completed reasoning and execution work, thereby reducing tail latency.

In summary:

- We introduce *agentic transactions* as a consistency abstraction for data-agent systems and identify their distinctive characteristics.
- We present Aegis, a transactional infrastructure for data-agents to provide consistent data views for concurrent execution.
- we propose ATCC, an adaptive concurrency control algorithm with learned phase-aware locking policy and priority-based lock scheduling, to mitigate contention.
- We integrate Aegis into openGauss [3] and conduct extensive experiments on agentic workloads against SOTA baselines. Experiments show Aegis improves the performance of agentic transactions up to XXX higher throughput and 90% lower tail latency.

2 Background and MOTIVATION

2.1 The Consistency Problem in Agent Task Processing

AI systems research is shifting from optimizing isolated model training toward AI-native datacenter infrastructures. Recent studies show that LLM-driven data processing is not a one-shot query-answering procedure, but a dynamic, multi-stage execution over data. Data-agent systems synthesize multi-stage workflows that plan, retrieve, transform, analyze, validate, and update data. In systems such as TAG [15], LOTUS [59], Palimpzest [48], and Deep Research [62], agents repeatedly interact with shared data and intermediate artifacts. Since later operation generation depends on earlier observation, task correctness requires a coherent task-level view across the entire operation sequence.

Data consistency has been treated as a first-order correctness requirement in decades of database research on systems that manage shared mutable data [14, 21, 27, 29, 30, 32, 34, 49, 55, 58, 88, 92, 93, 95]. Existing agent frameworks and harnesses improve workflow-level controllability, observability, and recoverability. Recent systems further introduce transactional semantics into agent execution, enabling partially executed tasks to be rolled back or recovered after failures [18, 54, 56, 79]. For example, BridgeScope [79] provides database-facing interfaces and ACID-aware primitives for LLM-database tasks, Atomix buffers and coordinates task-level effects, Saga-style systems [22, 28, 46] use compensation to recover long-running workflows, and version-based frameworks such as LangGraph [1] and AgentGit [44] support checkpointing, rewind, branching, or speculative execution. However, these mechanisms do not provide a consistent data view or strong isolation for the dynamically generated data operations that concurrent data-agent tasks perform over shared data. This limitation becomes especially

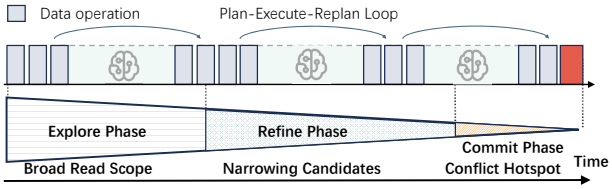

Figure 2: An execution illustration of agentic transactions.

critical in multi-agent architectures. When these agents operate on shared data, their dynamically generated reads and updates can interfere with each other, creating multi-agent concurrency conflicts that cannot be resolved by workflow rollback or agent-state recovery alone.

Serializability is the core correctness criterion for concurrent executions over shared data, and reliable data-agent reasoning requires task-wide data consistency. For data-agent systems, the key challenge is not only whether a failed task can be recovered after execution, but whether a task-wide consistent view can be maintained while read and write sets are generated online. This motivates our design of agentic transactions and concurrency control. The data operations of one agent task are treated as a single transactional unit that can commit or abort atomically, and concurrent accesses to shared data are controlled to preserve serializable execution.

2.2 Features of Agentic Transactions

Agentic transactions exhibit a coarse phase structure. Prior work characterizes data agent workloads as *agentic speculation*, where agents explore data, construct candidate solutions, and validate or execute selected outcomes [50]. Enterprise text-to-SQL and data-analysis benchmarks further show that realistic tasks involve metadata inspection, iterative querying, intermediate analysis, transformation, and validation [31, 38, 39, 41]. Accordingly, an agentic transaction typically progresses through three phases: *explore*, which performs broad read-heavy access to form an initial candidate space; *refine*, which issues more selective operations to revise or complete the plan; and *commit*, which validates the selected plan and performs a small number of critical updates.

zwx:question: see comments This phased evolution changes the concurrency conflicts over time. Early phases are broad and mostly read-heavy, and these reads determine which objects may later be updated. The commit phase is narrower, write-intensive, and more conflict-prone. Additionally, as illustrated in Section 1, reasoning and replanning introduce irregular gaps between data operations, and candidate generation makes access sets difficult to predict statically. Incorrect concurrency-control decisions can therefore degrade both system performance and service quality. Optimistic policies defer conflicts detection to the commit phase. A late abort discards accumulated reasoning, validation, refresh, replay, and retry effort, wasting completed work. Coarse pessimistic policies may overprotect early low-risk reads, reducing concurrency and throughput. Therefore, concurrency control for agentic transactions should not be limited to static policies. It should monitor runtime behavior and adaptively choose suitable protection based on transaction phase, access-set risk, and conflict feedback.

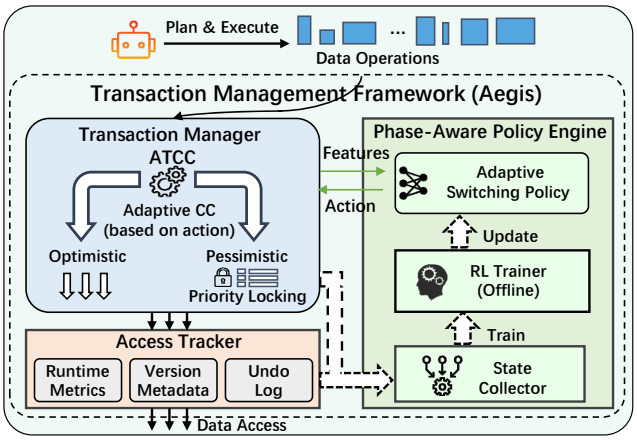

Figure 3: The overall architecture of Aegis.

3 Overview

3.1 Architecture

Aegis is designed as a transaction management framework placed between the data agent runtime and the underlying data stores. It receives data operations from the runtime, groups the operations of one agent task into an agentic transaction, and controls how the transaction accesses shared data.

As shown in Figure 3, Aegis consists of three main components. The Transaction Manager (TM) is responsible for the concurrency control (CC) of each agentic transaction. For every task, it creates a transaction context and maintains the transaction lifecycle, including begin, execution, adaptive switching, validation, commit, and abort. The context stores transaction private state, including the current CC action (Section 4.2), priority, read/write sets, *etc.* During execution, the TM invokes ATCC to apply the CC action, controlling each data operation executed optimistically or pessimistically.

The Access Tracker (AT) manages visibility and recovery for persistent objects updated by transactions, and maintains system-wide runtime statistics. Updates to large objects, such as files, checkpointed states, and disk-resident structured data, may have to be persisted before commit. Additionally, in-place updates would either expose uncommitted state or block concurrent readers. To ensure consistent data access and system concurrency, the AT is designed to maintain object-level version metadata and control the version access by transactions. zwx:question: see comments A transaction may create a new physical version while the last committed version remains visible to concurrent transactions. At commit, the AT advances each modified object to the newly committed version. After a failure, it uses the undo log to discard or revert uncommitted persistent updates. The AT also collects global contention signals, including abort rate, throughput, lock-queue length, and tail latency, for adaptive CC decisions and offline training.

The Phase-Aware Policy Engine (PE) guides the adaptive CC of ATCC. The policy engine uses the phase structure feature of agentic transaction to adjust the CC strategy over time, allowing Aegis to preserve concurrency early and reduce costly aborts near commit. Since workload patterns, data popularity, and contention levels may

change over time, the PE periodically retrains the policy offline using collected execution traces and refreshes the policy.

3.2 Workflow and Key features

Aegis executes each agentic task as a transaction. When a task first enters `Aegis`, the TM starts an agentic transaction and initializes its transaction context. Subsequent data operations are intercepted by the TM and executed according to the CC action. Each transaction starts with optimistic execution. During execution, Aegis periodically sends transaction and system-state features to the PE and updates the CC action based on the returned decision. When the policy identifies a higher conflict risk or abort cost, selected accesses are switched to lock-protected execution. At commit, `Aegis` validates accessed data versions and atomically makes committed updates visible. For persistent objects, the AT advances the visible version to the committed version. If the transaction aborts or fails before commit, uncommitted updates are discarded or recovered through the undo log. `Aegis` leverages three key features.

Phase-aware adaptive policy. Agentic transactions exhibit multiple reasoning and data access rounds, during which their access patterns and conflict risk evolve over time. The key observation behind `Aegis` is that agentic transactions are difficult to predict at the object level, but their execution often follows a phase-level progression. `Aegis` uses this phase structure to guide adaptive CC, preserving concurrency when accesses are uncertain and increasing protection when rollback becomes costly. This shifts the learning problem from operation prediction to phase inference estimation.

Contention-oriented selective locking. Locking every accessed object once a transaction becomes abort-prone would reduce aborts but suppress concurrency. Such a locking strategy lacks the flexibility to adapt to varying contention levels and execution phases, as it treats all accessed objects as equally risky and serializes low-contention accesses unnecessarily. `Aegis` instead uses contention-oriented selective locking, which adjusts the protected scope according to the inferred phase and current contention level. This allows `Aegis` only to protect accesses likely to cause costly aborts and retries while avoiding unnecessary blocking (Section 4.2).

Abort-cost-aware priority scheduling. Lock-based execution can introduce long blocking and deadlocks when conflicts occur, requiring the system to decide which transaction should be aborted under contention. In agentic workloads, the cost of aborting a transaction depends on the reasoning steps, tool calls, and data accesses already performed. Aegis reflects this asymmetry in lock scheduling by integrating abort-cost-aware, multi-level priority scheduling into the Wound-Wait locking scheme to prioritize critical transactions (Section 5.2.2).

4 Phase-Aware Adaptive Switching Policy

RL learns a control policy through trial-and-error interaction with the environment to maximize numerical rewards. An RL problem consists of states that describe the observed environment, actions available to the decision maker, a policy that maps states to actions, and a reward signal that evaluates the benefits of the actions. At each decision point, the RL agent observes the current state, selects an action according to its policy, receives a reward, and transitions to a new state. By learning from state-action trajectories spanning multiple decision rounds, RL can capture how earlier decisions affect subsequent states and long-term outcomes. To avoid ambiguity, we use agent exclusively for the LLM-based data agent and policy for the learned RL controller.

In `Aegis`, the CC decisions are made continuously throughout an agentic transaction. An earlier decision changes the transaction's locking scope and contention behavior, thereby affecting its subsequent execution and later policy decisions. This motivates us use RL to learn a phase-aware switching policy that optimizes the sequence of CC decisions.

In our formulation, the controlled environment consists of concurrently executing agentic transactions and the shared data objects they access. After completing a batch of operations, the policy observes the execution state of transactions together with system-level contention features. It then selects a CC action to determine whether a transaction should remain optimistic or introduce lock-based protection for selected data objects. Transaction execution result and overhead provide the reward feedback. The following subsections describe the design of state representation, action space, and reward function.

4.1 Evolving Execution State Representation

At each policy invocation, the PE constructs a state from phase-related features, contention-related features, and the transaction's current CC actions. Phase-related features characterize the transaction's execution progress, and contention features estimate the risk and cost of conflicts. These features describe transaction behavioral changes, enabling the policy to determine suitable CC actions.

Phase-related features. The following features are designed to characterize the execution progress of an agentic transaction.

- *Inter-round interval.* The elapsed time between consecutive data operations. It is used to approximate the reasoning and tool-execution work and cost. Longer intervals indicate more costs and higher retry costs.
- *Access-set growth.* The increment of the read/write set size. Read-dominated growth indicates continued exploration, whereas increasing writes indicate progression toward updates and commit.
- *Access-set overlap.* The overlap between data objects accessed in consecutive rounds. Low overlap indicates that the agent continues to explore new objects, while high overlap suggests that that the access scope is stabilizing.
- *Execution progress.* The number of of completed rounds and operations, and proportion of writes issued in recent rounds. Indicates whether the transaction has transitioned from the refine phase to the commit phase.

Contention indicators. The following features are designed to evaluate the transaction's contention level and abort probability.

- *Hotspot access ratio.* The proportion of hotspot records in the read/write sets. Transactions are more likely to be aborted due to conflicts when accessing more hotspot objects.
- *Blocking time.* The total time a transaction spends waiting for locks. A longer blocking time indicates higher data contention and risk of starvation.
- *Local conflict history.* The history includes the number of conflict-induced restarts and recent observed conflicts (*e.g.,* validation

Table 1: Contention-driven actions in Aegis

Action	Description and Rationale	
Remain in OCC Mode	Continue under OCC	
Locking Strategies	Hot Read set Hot Write set	Cold Read set Cold Write set

failures or lock preemptions). Repeated failures indicate high persistent contention and risk of another optimistic abort.

- *Global contention metrics.* TM metrics, including the abort rate, throughput, average and tail latency, and lock-queue length. These features provide the system-wide contention.

4.2 Contention-Aware Switching Policy

To achieve high performance when executing agentic transactions, the action space needs to enable elastic CC switching. Table 1 shows the action space of Aegis. All transactions start in OCC because they are predominantly read-only in the explore phase. As execution progress, both conflict risk and accumulated restart cost may increase. The policy then introduces locks on selected accesses to reduce the likelihood of a costly abort.

Locking actions are defined along two dimensions: access type and object contention. Access type distinguishes reads from writes, while object contention classifies accessed objects as hot or cold. These dimensions capture different locking trade-offs. Locking writes reduces write conflicts, whereas locking reads prevents concurrent updates from invalidating observed values but introduces additional blocking. Similarly, locking hot objects can effectively mitigate concentrated conflicts, while locking cold objects may restrict concurrency with limited benefit. The four access classes can be selected independently. Based on execution progress, contention, and restart cost, the policy determines which classes should use locks. This design enables selective pessimistic execution without encoding exact object identifiers or placing all accesses of a transaction under locking.

4.3 Cost-Aware Reward Design

The CC action introduces a trade-off between restart cost and locking overhead. Optimistic execution avoids blocking but risks discarding completed data operations and agent-side work upon abort. Locking reduces this risk but may increase waiting and restrict concurrency. Thus, the reward is designed to balance these effects.

At each decision step t, Aegis first computes the restart cost $C_{\text{restart}}(T,t)$, which consists of operation and agent-side cost.

$$C_{\text{restart}}(T,t) = \omega_1 \overline{C}_{\text{op}}(T,t) + \omega_2 \overline{C}_{\text{agent}}(T,t) \tag{1}$$

Operation cost is measured from data-operation service time excluding lock waiting. Agent-side cost includes reasoning and tool invocation, measured directly or approximated using inter-round agent time. These two cost are normalized from training traces, preventing differences in units and magnitude from causing one component to dominate the reward.

The reward r_t is defined in Equation 2.

$$r_t = \beta_1 \mathbf{1}_{\text{commit}} - \beta_2 \left(1 + C_{\text{restart}}(T,t) + \beta_3 C_{\text{retry}}(T,t)\right) \mathbf{1}_{\text{abort}} - \lambda C_{\text{lock}}(T,t) + \eta \Delta P_{\text{sys}}(t). \tag{2}$$

A commit receives a fixed reward, while an abort incurs a penalty with the current restart cost and the work discarded by previous fails. $C_{\text{lock}}(T,t)$ captures lock waiting and newly acquired locks, penalizing the overhead of pessimistic execution. $\Delta P_{\text{sys}}(t)$ summarizes changes in agent-task throughput, end-to-end latency, and conflict-abort rate. These terms guide the policy to introduce locks only when the expected reduction in abort cost outweighs the resulting concurrency overhead.

4.4 Low-Latency Policy Invocation

Invoking the RL policy takes milliseconds and can noticeably increase execution latency. Although fine-grained state features improve decision quality, invoking the policy during transaction execution may offset its benefit. To address this, Aegis compiles the learned policy into a policy table for runtime decisions. Since transaction states are continuous and high-cardinality, they cannot directly serve as table indices. Aegis maps each transaction state to a compact *state key*. Categorical features are encoded directly, while numerical features are bucketized using logarithmic or quantile-based boundaries derived from training traces. Each state key is paired with the policy-selected locking action, and the resulting entries are grouped by action. Within each group, weighted (k)-medoids retains a small set of representative keys, using key frequencies as weights. In Aegis, we set k = 1 by default for fast table look up. During runtime, a transaction constructs its state key, traverses the action groups, and selects the representative key-action with the smallest weighted distance. Its runtime cost consists only of incremental feature maintenance, state-key construction, and in-memory table lookups.

Selective refinement. Discretization may map distinct execution states to similar table keys, reducing decision fidelity. Optionally, Aegis supports directly invoking the policy when the nearest key is insufficiently similar and policy inference can overlap an agent-side reasoning or tool-execution interval. The result refines the current action if it arrives before the next policy-table lookup.

4.5 Policy Training

Aegis periodically retrains its switching policy offline to adapt to evolving access skew and contention in agent workloads. Each transaction produces a trajectory of state, applied action, reward, and next-state transitions collected at policy control points. Aegis uses PPO to optimize the policy over these trajectories. During training, PPO explores alternative locking actions to collect reward feedback. During deployment, the exploration is disabled to ensure stable and predictable runtime decisions. After training, the updated policy, state-key encoder, and policy table are installed to Aegis.

A policy retraining is triggered after collecting N_r new transaction trajectories. A smaller N_r responds more quickly to workload changes but increases training cost and may produce unstable updates from insufficient samples. A larger N_r provides more stable training data and lower update overhead but may leave the deployed policy stale for longer. Aegis adjusts the refresh interval

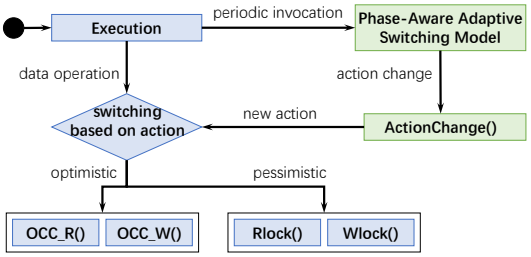

Figure 4: The execution flow and function calls of adaptive concurrency control.

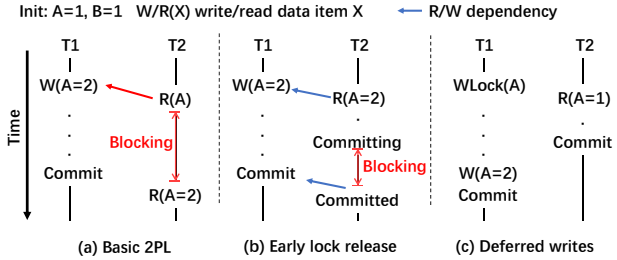

Figure 5: Transaction execution under different CC mechanisms: (a) basic 2PL, (b) early lock release, and (c) deferred writes.

using the average nearest-key distance and system-level performance metrics. Increasing lookup distance, conflict-abort rate, or tail latency shortens the interval, while stable lookup distances and performance allow the interval to grow.

5 Adaptive Concurrency Control

This section presents the CC protocol of Aegis. The TM creates a transaction context for each agent task and processes its data operations through ATCC. As shown in Figure 4, transactions execute in a hybrid mode, with the current CC action determining which accesses remain optimistic and which are lock-protected (Section 5.2). The TM periodically consults the adaptive policy to update this action. Upon a switch, Aegis validates prior accesses and applies the new protection scope (Section 5.3). All transactions then follow a unified validation protocol to commit updates (Section 5.4).

5.1 Metadata for Fine-Grained Adaptation

Aegis maintains DRAM metadata for each object and transaction to enable adaptive execution. For each object, the AT atomically looks up or creates a descriptor, assigns it a logical object descriptor when it is first accessed. The descriptor records its physical address, and initializes its version to zero. Concurrent accesses to the same object share one descriptor. Each descriptor also contains a read-lock count (*R_count*), a reader list (*R_list*), a write-lock owner (*W_owner*), and a priority-ordered lock queue (*lock_queue*). The write-lock owner includes a committing *latch*. After a transaction has acquired all required write locks and passed validation, it sets the committing latches, entering a non-preemptive committing state that prevents lock preemption (Section 5.4).

For each agent task, the TM creates a transaction context (*ctx*) identified by a unique identifier *tid*. The context records the transaction status (running, commit, or abort), start timestamp (*startTS*), current phase (*phase*), CC action (*action*), and scheduling priority (*priority*). It also tracks the read and write sets (*RS* and *WS*), their hot subsets (*HRS* and *HWS*), the locks currently held, and pending lock request. An RS entry references the logical object identifier and observed version. A WS entry additionally records the base version and a reference to the private update stored in memory or on disk. The context also accumulates contention and execution statistics used by the phase-aware policy. All contexts are maintained in a globally accessible array indexed by *tid*.

5.2 Hybrid Execution

In Aegis, transactions begin with the optimistic and periodically adapt their access modes to evolving execution conditions. The current action determines which access remain optimistic or pessimistic. Since actions are transaction-specific, concurrent transactions may apply different access modes to the same object.

For each access, the TM controls the access mode through the AT, which atomically looks up or creates the object descriptor. A read first checks the transaction's local read and write sets to provide consistent read and read-your-writes semantics. Otherwise, an optimistic read obtains the object through the descriptor, and records the observed metadata in RS (non-blocking reads, Section 5.2.1). A pessimistic read additionally acquires a read lock before access. All writes are stored in the write set and remain private until commit. An optimistic write records the base version and private update in WS without acquiring a write lock. A pessimistic write acquires the write lock before recording the update, thereby establishing exclusive write ownership. In both cases, the descriptor's committed address and version remain unchanged until commit.

5.2.1 Non-Blocking Optimistic Execution via Deferred Writes. In Aegis, optimistic and pessimistic execution may coexist on the same object. However, long lock-holding times can cause head-of-line blocking and degrade responsiveness. As shown in Figure 5a, transaction T_1 pessimistically writes A, while T_2 optimistically reads it. Once T_1 acquires the write lock and updates it, T_2 remains blocked until the lock is released.

Early lock release [26, 33] (Figure 5b) reduces this delay by releasing locks before commit. Once T_1 completes its updates, it releases the lock so that T_2 can read the new value. Because this value is uncommitted, T_2 is not permitted to commit before T_1 does to avoid dirty reads. This commit-order constraint remains costly in agentic workloads. even if T_2 finishes quickly, it may still wait for the long-running T_1 to commit.

To avoid long blocking, ATCC adopts *deferred writes* (Figure 5c). A pessimistic writer acquires the write lock to serialize conflicting writes but buffers its update in the local write set instead of immediately updating the object in place. Thus, T_2 can read the lastest committed value ($A = 1$) and commit without waiting for T_1. More generally, optimistic reads ignore write-lock ownership and access the latest committed version, while optimistic writes update only their local write sets. Deferred writes therefore prevent dirty reads while eliminating the commit-order constraint of early lock release.

Algorithm 1: Priority-Based Pessimistic Locking in Aegis

```
Input: transaction context ctx
Output: return true if success; false otherwise
Function RLock(Object_descriptor obj):
    if W_owner == tid or tid ∈ reader_list then
        return; //already get the lock.
    if W_owner ≠ INVALID then
        writer = ctx_arr[W_owner.wid];
        if writer.priority < ctx.priority then
            Abort(writer);
        else
            Insert(lock_queue, ctx, Read) and Wait;
            //wait until get the lock or abort by other txn.
    reader_list.add(ctx.tid);
    R_count += 1;
    Insert(obj, ctx.RS/HRS);
Function WLock(Object_descriptor obj):
    if W_owner == tid then
        return;
    if R_count ≠ 0 then
        foreach R_tid in reader_list do
            reader = ctx_arr[R_tid.wid];
            if reader.priority > ctx.priority then
                Insert(lock_queue, ctx, Write) and Wait;
        foreach reader in reader_list do
            Abort(reader);
            R_count -= 1;
    if W_owner is INVALID then
        W_owner = tid;
    else
        writer = ctx_arr[W_owner.wid];
        if writer.priority < ctx.priority then
            Abort(writer);
            W_owner = tid;
        else
            Insert(lock_queue, ctx, Write) and Wait;
    Insert(obj, ctx.WS/HWS);
```

5.2.2 Priority-Based Locking. As agentic transactions execute, Aegis may protect selected accesses with pessimistic locks to reduce validation failures on contended objects. To resolve conflicts among pessimistic access, Aegis uses *priority-based locking*, motivated by the highly asymmetric cost of transaction aborts. Unlike existing priority-based CC schemes [2, 4, 25, 69, 86], which typically use a fixed priority throughout the transaction lifetime, Aegis dynamically re-evaluates transaction priority as execution proceeds (Section 5.3.1). This allows conflict resolution to adapt to the changing cost of aborting each transaction.

Algorithm 1 shows the locking procedures. For reads, the RLock() function is provided for transactions to acquire read locks and resolve concurrent read-write conflicts. The transaction is granted the lock immediately if it already holds the object, and otherwise resolves read-write conflicts using priority-based Wound-Wait: a lower-priority transaction waits, whereas a higher-priority transaction wounds the conflicting writer. For writes, the WLock() function handles both write-write and write-read conflicts using the same rule. Once the lock is acquired, the object descriptor is added to the read and write sets, and the modification is marked as private and deferred to the commit phase to apply updating by delayed write apply.

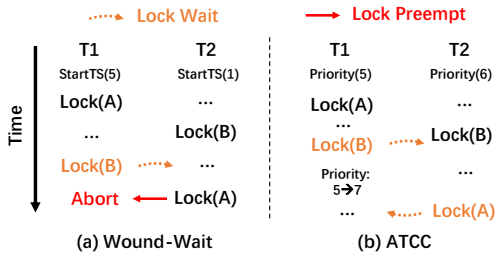

Figure 6: Deadlock caused by transaction priority changes when using the Wound-Wait algorithm.

In Aegis, the priority is defined by the pair $(p, startTS)$, where p is the current priority and $startTS$. Specifically, we define $T_1 > T_2$ if $p_1 > p_2$, or if $p_1 = p_2$ and $startTS_1 < startTS_2$. The case $startTS_1 = startTS_2$ cannot occur, as 'startTS' is assigned atomically.

5.3 Runtime Strategy Adaptation

In Aegis, transactions periodically adjust their priority and locking actions to reduce aborts.

5.3.1 Priority Change and Deadlock Prevention. As discussed above, Aegis assigns each transaction a non-decreasing priority that reflects both its accumulated work and its risk of starvation.

$$\text{Prio}(T) = \left\lfloor \alpha \frac{\text{Op}(T)}{\Delta_o} \right\rfloor + \left\lfloor \beta \frac{\text{Agent}(T)}{\Delta_a} \right\rfloor + \lfloor \lambda \, \text{Retry}(T) \rfloor + \left\lfloor \rho \frac{\text{Wait}(T)}{\Delta_b} \right\rfloor \quad (3)$$

As shown in Equation 3, $\text{Op}(T)$ measures the completed operation cost, $\text{Agent}(T)$ captures reasoning, tool invocation, and other agent-side work costs, $\text{Retry}(T)$ counts previous aborts, and $\text{Wait}(T)$ tracks the cumulative blocking time. All parameters are learned during RL training, enabling workload-aware prioritization with lightweight runtime computation. Because all components are cumulative within single execution, the priority never decreases.

Classic Wound–Wait resolves conflicts using a fixed priority order, e.g., unified $startTS$. As depicted in Figure 6a, transaction T_2 has a higher priority than T_1 since it starts earlier. Therefore, T_1 waits when requesting B from T_2, whereas T_2 wounds T_1 when subsequently requesting A. Since the priority order is fixed, wait edges always point from lower-priority to higher-priority transactions, and the wait-for graph remains acyclic.

In contrast, dynamic priorities can invalidate this property. In Figure 6b, T_1 is initially blocked by T_2 on B. After T_1's priority is raised, T_2 may in turn become blocked by T_1 on A. This results in a circular dependency, leading to a deadlock.

Aegis eliminates such stale wait edges by reevaluating a waiting transaction's pending lock request whenever its priority increases. The TM first removes the transaction's pending request and then repeats conflict resolution using the updated order. The transaction either wounds the now-lower-priority holder or is reinserted into the queue as new-lower-priority waiter. This ensures that every wait edge follows the current priority order, keeping the wait-for graph acyclic. Because each transaction has at most one pending

Algorithm 2: Functions of Validation

```
   Input: transaction context ctx
   Output: return true if success; false otherwise
1  Function ActionChange(LockAction ac):
2      //Lock the HRS or RS, HWS or WS based on the lock action.
3      foreach obj, read_ver in txn.HRS/RS do
4          if read_ver == obj.curr_ver then
5              RLock(obj); //Lock the obj or abort.
6          else
7              Abort(ctx); The obj has been updated by other txn,
                abort.
8      foreach obj in txn.HWS/WS do
9          WLock(obj); //Lock the obj or abort.
10 Function Validation():
11     foreach obj in txn.WS do
12         WLock(obj); //Lock the obj or abort.
13     foreach obj, read_ver in txn.RS do
14         if reader_list.Find(ctx.tid) then
15             continue; The obj is protected by the RLock.
16         while IsCommitting(obj) do
17             Abort(ctx); //The obj is updating by other txn, abort.
18         if read_ver ≠ obj.curr_ver then
19             Abort(ctx); The obj is updated by other txn, abort.
20     foreach row in txn.WS do
21         SetCommitting(row);
```

lock request, reevaluation touches only one queue entry and incurs little overhead.

5.3.2 Locking Action Change and Retroactive Validation. An action change may require previously optimistic accesses to become lock-protected. To preserve serializability (Section 5.5), the TM identifies such objects, retroactively validates their earlier accesses, and acquires locks. The ActionChange function (Alg. 2, lines 1–9) describes this procedure. For each newly protected read, the TM atomically checks the version recorded in read set and attempts to acquire a read lock on the corresponding descriptor. A version mismatch causes the transaction to abort, while a lock conflict is handled by the priority-based locking protocol. For each newly protected write, the TM similarly acquires or upgrades to a write lock. The request may wait for a higher-priority or committing holder, or wound a lower-priority holder. If a request waits, the TM revalidates the recorded version after it wakes and before granting the lock. Subsequent accesses follow the new action.

==========todo rewrite line==========

5.4 Unified Commit Protocol

To enable correct mixed optimistic and locking-based pessimistic concurrency control, Aegis applies a unified commit validation procedure to all transactions to ensure serializability (Alg. 2, lines 10–21). Similar to Silo [74], validation first locks the write set and then validates the read set. Specifically, Aegis uses priority-based locking to lock all objects (descriptors) in the write set and detect write-write and read-write conflicts across transactions with varying priorities. It then validates the read set. Objects already protected by RLock need no further validation, whereas optimistically read objects are checked by version validation rather than by acquiring read locks (similar to Silo).

After validation, the transaction enters the non-preemptive committing state by setting the committing `latch` through SetCommitting. Finally, the transaction obtains a commit version installs the buffered writes into shared tuples. After commit or abort, the transaction enters a cleanup phase that releases the read and write locks (including committing latch). Waiting transactions are then granted access according to the priority-based lock policy.

5.5 Correctness Proof

Although Aegis combines optimistic reads, lock-based access, deferred writes, dynamic operation changes, and priority-based scheduling, it still guarantees that conflicts can be serialized. The claim follows from four properties of the unified commit protocol (Alg. 2).

First, all writes are ordered by WLock. Hence, conflicting writers cannot both commit without an explicit lock order. Second, every read-write conflict is either ordered by locking or detected by validation: locked reads participate directly in lock-based conflict control, while optimistic reads must pass version validation before commit. Thus, a reader cannot commit after observing a stale version overwritten by a concurrent committed writer. Chunwei: Merge the two paragraph or list as three paragraphs for the three points. Third, deferred writes ensure that updates remain private until commit. After validation, the transaction marks each write-locked tuple as committing before installing buffered writes, so other transactions can observe either the old committed version, the new committed version, or a retry condition, but never an uncommitted or partially installed state. Fourth, if a transaction changes an access from optimistic to locked execution, retroactive validation ensures that the earlier optimistic observation is still valid; otherwise, the transaction aborts.

Priority-aware scheduling does not affect correctness, since it changes only the order in which waiting transactions acquire locks and does not allow any transaction to bypass locking, validation, or commit ordering. Therefore, every conflict among committed transactions is ordered consistently with commit-time serialization points, and the resulting execution is conflict-serializable.

5.6 Discussion

In Aegis, an action change never involves releasing locks. Only agentic transactions with a high abort risk and large retry cost choose to acquire locks. Even when conflicts become rare, releasing these locks would expose the previously read data to modifications, and any resulting aborts would incur an unacceptable retry cost. Moreover, most transactions can rely on *deferred writes* to read data optimistically without being blocked by locks. Only those agentic transactions that actually encounter conflicts need to acquire locks to ensure conflict-free execution. Therefore, Aegis does not adopt the release lock action. Once a transaction has escalated to a locking action, it retains that action until it is either committed or aborted.

6 Recovery

7 Evaluation

In this section, we evaluate the performance of Aegis.

Implementation. Aegis is implemented based on openGauss-MOT [3]. openGauss is an open-source industrial-grade database system, and openGauss-MOT is an in-memory transactional storage engine designed for high-performance OLTP workloads. Unlike most academic in-memory database prototypes [19, 76, 77, 86, 87], which execute transactions in a stored-procedure mode with fixed access patterns, Aegis leverages the SQL interface provided by openGauss-MOT to support flexible query processing. This flexibility allows Aegis to seamlessly integrate with agentic transaction scenarios, which contain various query types, enabling adaptive concurrency control in real-world operational environments.

Testbeds. Our experiments run on a server equipped with Intel® Xeon® Platinum 8360H CPUs (each with 24 physical cores, 33 MiB LLC, clocked at 3.0 GHz), 768 GiB DDR4 DRAM, and a 2TB NVMe-SSD. The operating system is CentOS 7.9 (Linux release 7.9.2009, kernel 3.10). To ensure stable performance results, all worker threads are pinned to a single NUMA node, thereby eliminating cross-NUMA memory access overhead.

Competitors. In our experiments, we compare Aegis with several representative CC schemes to evaluate the performance:

- Wound-Wait: A variant of 2PL where a transaction preempts the lock if it is older than the lock holder and waits if younger.
- Silo [74]: A representative implementation of OCC and serves as a baseline for high-performance in-memory OLTP systems.
- MOCC [77]: A hybrid concurrency control scheme that selectively acquires read locks on hot records during execution and enforces a canonical lock-ordering protocol to avoid deadlocks.
- Plor [19]: A hybrid CC protocol that transactions acquire locks before writes but perform reads optimistically. Conflicting reads are validated in the commit phase.
- Polaris [86]: An OCC variant that supports transaction priorities via lightweight reservations.
- Polyjuice [76]: A learning-based CC framework that models each transaction operation as an execution state and learns a mapping from states to concurrency control actions.

Workloads. Experiments use three benchmarks, including YCSB [8], TPC-C [6] and a simulated ticket reservation workload. In YCSB, each query accesses a single record, and the contention level is controlled via a Zipfian distribution. To make it a transactional benchmark, we wrap 10 operations within a transaction and use one table with 10 columns and 1,000,000 rows. TPC-C is an industry-standard benchmark for evaluating online transaction processing (OLTP) databases. Only *Payment* and *NewOrder* are used in our evaluation, since several baselines only support these types. Inspired by the characteristics of agentic workloads, we adapt YCSB and TPC-C to emulate agent-like interaction patterns, and refer to the resulting benchmarks as *Agentic-like YCSB* and *Agentic-like TPC-C*. Clients issue agentic transactions with random inter-operation delays (uniformly distributed in 1-20 ms) to model the agent's extra processing. To model abort handling, when an agentic transaction aborts, we delay its retry by an additional random reasoning time (500 ms-5 ms) to reflect plan reconsideration before re-execution.

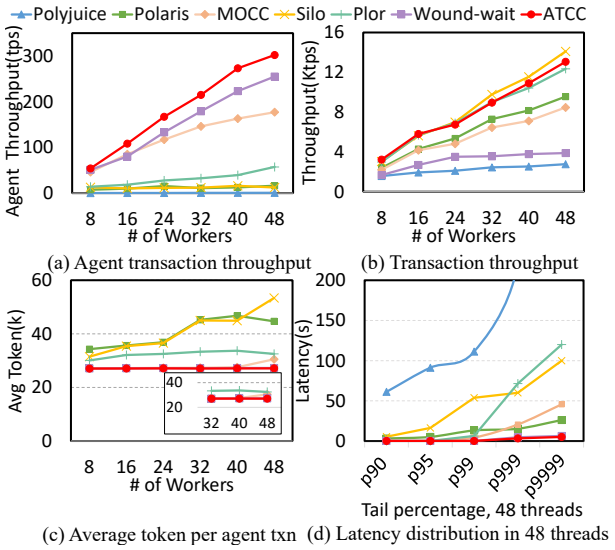

Figure 7: Results when varying the number of workers (YCSB Medium-Contention).

Inspired by [36, 83], the simulated ticket-booking workload represents an agentic Flight-Booking scenario. In this workload, we implement an *agent* using a lightweight *proxy* that communicates with the database through SQL interfaces and connects to ChatGPT-4.0 to acquire reasoning and natural language understanding capabilities. The agent operates on a relational schema (including Routes, Prices, Aircraft, and Seats) via a multi-turn ReAct workflow. More details on the workload and an example are shown in Section B.

Specifically, in our testing, 80% of clients issue agentic transactions, while the remaining 20% continuously execute stored-procedure transactions in the background to represent conventional workloads. To prevent frequent aborts due to accessing hot records, stored-procedure transactions retry after a shorter backoff (10-30 ms). Unless otherwise noted, we report end-to-end transaction latency measured from the initial submission until the transaction eventually commits (including all retries and injected delays).

To quantify the overhead of LLM interactions, we estimate the average token consumption per transaction, denoted as T_{avg}. Based on profiling the Flight Booking Agent Workload, we set the average tokens per SQL operation $\omega \approx 2703$. T_{avg} is calculated as $T_{avg} = (1 + R_{abort}) \cdot N_{ops} \cdot \omega$, where R_{abort} is the abort rate and N_{ops} is the number of operations per transaction.

7.1 Agentic-like YCSB Results

We first analyze the performance of all the concurrency control algorithms on the YCSB workload with three different variations:

- Low Contention: 95% reads - 5% writes with a uniform access distribution (θ = 0.0).
- Medium Contention: 90% reads - 10% writes with a hotspot of 10% tuples that are accessed by ~ 50% of all queries (θ = 0.7).
- High Contention: 50% reads - 50% writes with a hotspot of 10% tuples that are accessed by ~ 75% of all queries (θ = 0.99).

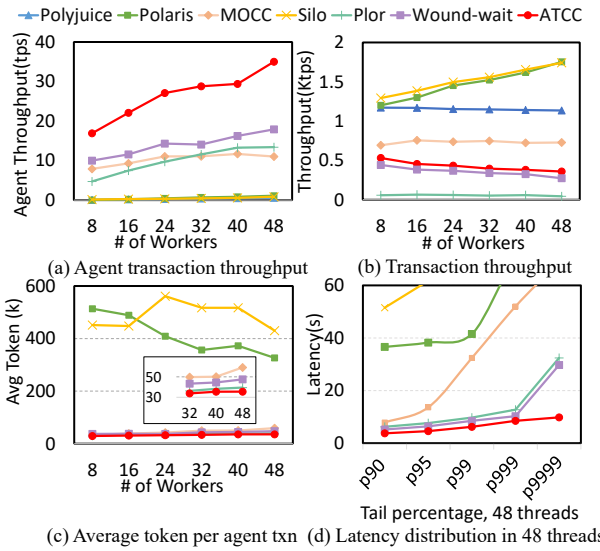

Figure 8: Results when varying the number of workers (YCSB High-Contention).

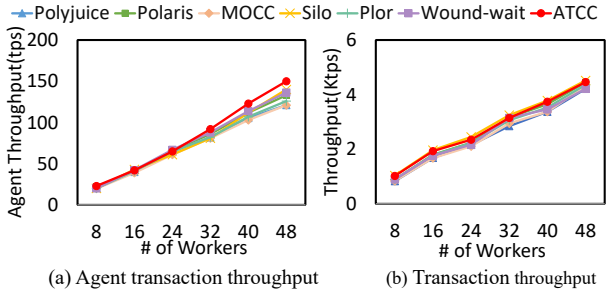

Figure 9: Results for the different CC algorithms (TPC-C with 100 warehouse).

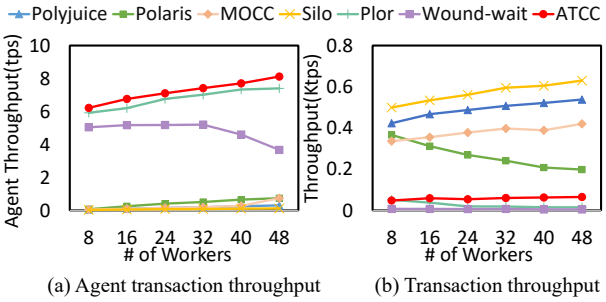

Figure 10: Results for the different CC algorithms (TPC-C with 1 warehouse).

Medium contention. When contention increases, however, the ways these algorithms handle conflicts begin to significantly affect the performance of long-lived agentic transactions.

As shown in Figure 7, all systems continue to scale with increasing worker threads under medium contention, but the performance of agentic transactions degrades to varying degrees depending on each CC scheme's conflict handling and ability to prioritize agentic workloads. Silo and Polaris, relying on optimistic validation, suffer from frequent aborts as agentic transactions repeatedly read stale versions of hot keys, as they are repeatedly invalidated by faster stored procedures. While the total throughput remains high, this is a misleading metric achieved by cannibalizing agents, leading to the retry loop, and causing agent throughput to collapse (Figure 7a). Wound-Wait improves agent throughput via preemptive locking but degrades the overall system performance due to the indiscriminate blocking of short background transactions. MOCC and Plor fall between these two extremes but fail to fully resolve the trade-off: MOCC releases locks prematurely, exposing previously read data to concurrent updates, leading to aborts. Plor avoids concurrent updates using write locks, but in read-intensive workloads, many agentic transactions are still aborted by shorter transactions. Polyjuice, optimized for static stored procedures, struggles to adapt to the random access, further impacting performance.

`Aegis` achieves high throughput by adaptively switching hot data access to pessimistic mode. Crucially, by leveraging *deferred updates*, agentic read locks do not block the validation of concurrent stored procedures. This strategy achieves a high agentic throughput at 48 threads, higher than Polaris by 18.1× and Polyjuice by 76.9×, while reducing token costs by ∼ 16% compared to Plor. Additionally, `Aegis` remains stable with tail latency, lowering by 5× compared to Polaris.

High contention. Figure 8 presents throughput over a spectrum of thread numbers in a high contention workload. When contention becomes high, hot data items are accessed by many transactions concurrently. Silo, Polaris, and Polyjuice maintain high *total* throughput but suppress agentic transactions to near-zero levels. Figure 8(a) highlights `Aegis` as the only robust solution, outperforming MOCC and Wound-wait by approximately 3.1× and 2.2× in agentic transaction throughput, respectively. While other baselines drop to near-zero throughput due to the retry loops. This performance advantage stems from `Aegis`'s proactive locking strategy on hot records. Although this protective mechanism reduces short transaction throughput compared with purely optimistic schemes, it ensures fairness and economic viability.

Crucially, `Aegis` keeps the abort rate below 30%, reducing token costs by over 15× compared to Silo/Polaris (≈ 35k vs. 500k+). `Aegis` exhibits superior tail latency control with fewer retries. Competitors suffer from unbounded latency growth due to uncontrolled retries, `Aegis` commits the vast majority of agentic transactions with bounded delay, lowering the P99.99 latency by approximately 3× compared to Wound-wait.

7.2 Agentic-like TPC-C Results

We now evaluate the concurrency control protocols using the Agentic-like TPC-C workload under two different contention levels by using 1 and 100 warehouses. The number of warehouses in TPC-C determines both the size of the database and the amount of concurrency. The warehouse is the root entity for almost all of the tables in the database. We first run TPC-C with 100/48 warehouses to simulate a low-contention environment rich in parallelism and then with 1 warehouse to represent high contention where transactions compete heavily for the same set of records.

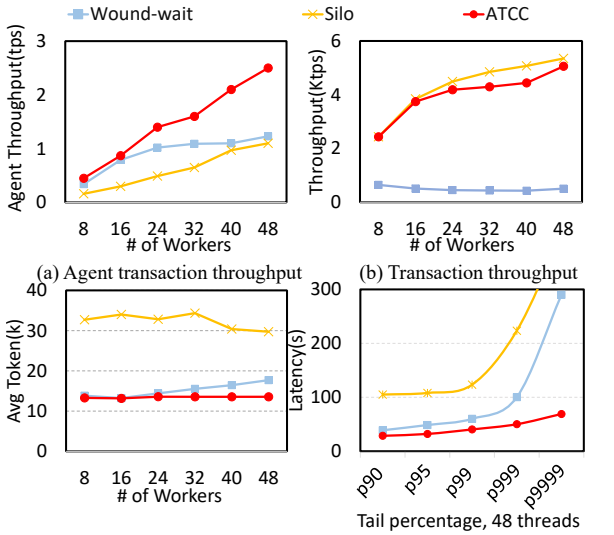

Figure 11: Performance comparison on the Flight Booking agentic workload.

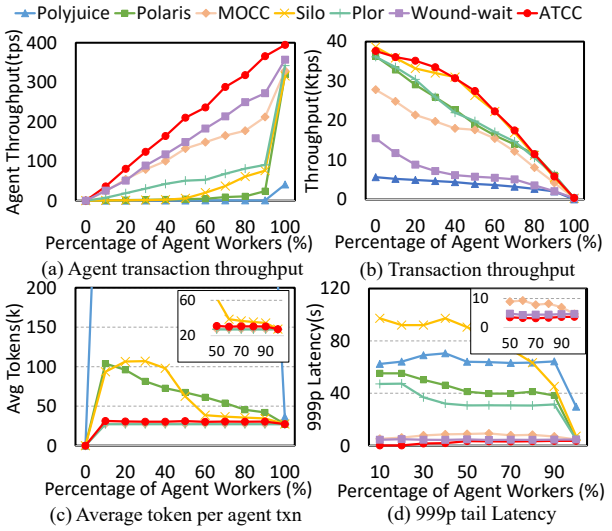

Figure 12: Performance with varying interactive client proportions (YCSB Medium-Contention).

Low Contention. Figure 9 shows the throughput results for the 100-warehouse case. When contention is low, all protocols scale almost linearly with increasing worker threads, consistent with the YCSB-Low-Contention results. Unlike YCSB's random accesses, each TPC-C transaction has a fixed sequence of operations and stable data dependencies. This determinism allows **Polyjuice**, which uses an offline pre-trained policy, to allocate nearly optimal concurrency-control strategies for each operation, thus exhibiting excellent scalability similar to Silo.

High Contention. Figure 10 presents the results for the 1-warehouse configuration, where contention is high due to intense competition for the same records. Silo and Polaris suffer sharp throughput degradation of agentic transactions due to validated aborts. Plor exhibits the lowest performance, bottlenecked by both write blocking and read-conflict invalidations. MOCC and Polyjuice achieve relative stability using explicit hot-term locking and deterministic policy learning, respectively, but they lack specific mechanisms to prevent long-running proxies from running out of resources.

In contrast, `Aegis` ensures the viability of agentic transactions by actively protecting their read and write sets. By enforcing pessimistic isolation on hot records, `Aegis` achieves an agentic throughput 2.2× higher than Wound-Wait and over 10× higher than other baselines. This confirms that under pathological contention, targeted protection is the only viable strategy to preserve interactivity stability.

7.3 Agentic Workload Performance

In this experiment, we evaluate how `Aegis` performs on an agentic ticket-ordering workload. Since MOCC, Plor, Polaris, and Polyjuice do not support standard SQL interfaces, we just compare `Aegis` with Wound-Wait and Silo under the same code-base. Figure 11 presents the performance results under the Flight Booking workload (80% agentic mixed).

As shown in Figure 11(a), `Aegis` achieves a peak agent throughput of ~2.5 tps, outperforming Wound-Wait by 2× and Silo by 2.5×. Crucially, this gain does not compromise background tasks. In Figure 11(b), `Aegis` maintains a total throughput (~5k tps) comparable to Silo. Employing adaptive locking with priority-aware scheduling, ATCC effectively protects long-running agents from starvation without inducing system-wide gridlock. Conversely, Silo achieves high total throughput only by starving agentic transactions, rendering it practically unusable for agent-centric applications. Similarly, Wound-Wait's indiscriminate locking triggers severe contention and preemptive aborts of agentic transactions, causing total throughput decreased by nearly 10× compared to `Aegis`.

Figure 11(c) and (d) highlight the economic implications and tail latency. Silo incurs prohibitive costs (~33k tokens) due to pathological retry loops, in which agents repeatedly waste tokens on reasoning only to fail validation. `Aegis` eliminates this waste via adaptive locking, reducing token costs by approximately 60% (~13k). Regarding tail latency, `Aegis` maintains a bounded P99.99 latency of ~70s. This represents a 4× reduction compared to Wound-Wait (~290s), which suffers from significant lock queuing delays and aborts under high concurrency.

7.4 Ablation Study

Real-world deployments exhibit dynamic shifts between interactive and background-heavy phases. A practical CC scheme must therefore remain robust as the mix of agentic and stored-procedure clients shifts over time. To evaluate robustness under such fluctuations, we vary the proportion of agentic workers from 0% to 100% (48 threads total) using the YCSB medium contention workload.

As illustrated in Figure 12, while total throughput naturally declines as long-running transactions replace short procedures, `Aegis` demonstrates exceptional robustness. It achieves nearly linear growth in agentic throughput, maintaining a negligible abort rate (~ 0.2%) across all mix ratios. By dynamically switching hot

agentic operations to lock-protected mode while keeping background tasks optimistic, Aegis maintains minimal token costs (~27k) and prevents latency spikes.

In contrast, baselines employing uniform conflict resolution fail to protect agent transactions. When agentic clients are few (10% - 50%), long-running agents are repeatedly invalidated by high-frequency background updates. This inefficiency manifests as a rapid escalation in costs: Silo's token consumption spikes to ~100k - 3.3× higher than Aegis, while Polaris suffers from tail latency spikes nearly 10× worse than Aegis.

8 Related Work

Interactive Transactions and Agentic Workloads. Interactive transactions, wherein clients issue multiple database commands to be executed within a single transaction, are fundamental to modern applications [13, 37, 57]. Inspired by systems like System R [11] and Ingres [66, 70, 78], modern database systems [9, 12, 20, 23, 65, 69, 75, 82, 88, 90, 92**? ? ? ? ? ? ? ? ? ?**] have introduced interactive interfaces to support various applications. Traditional interactive transactions exhibit think time and runtime-dependent access patterns. Agentic transactions share these properties but introduce additional challenges/differences:

Although long think time and runtime-dependent access sets also appear in conventional interactive transactions, data-agent systems make these properties the common case and combine them with explicit reasoning state, intermediate artifacts, and dynamically generated internal operators. why?

The model is more general than existing adaptive concurrency control algorithms, such as Polyjuice [47] and bLDSF [45]. The former only maps data access IDs to pipeline-wait actions, and the latter only maps dependency set sizes to priority-related wait actions.

Learning-Based Transaction Management. Machine learning techniques have been applied to enhance the performance and adaptability of database transaction management. [64] analyzes potential conflicts from SQL predicates and assigns high-conflict transactions to dedicated FIFO queues for serial execution. However, it requires the complete set of SQL statements to be known before execution, making it inapplicable to agentic workloads where queries are generated incrementally by LLMs. Polyjuice [76] treats CC as a fine-grained policy that maps per-operation execution states to actions and trains offline to obtain an optimized state-action policy table. While effective for deterministic stored procedures, its static policies fail to adapt to the stochastic, non-deterministic access patterns of interactive agents, leading to policy misalignment and performance degradation. Aegis employs a reinforcement learning framework explicitly designed for agentic stochasticity. Rather than relying on static templates or pre-declared query sets, Aegis analyzes real-time system contention and phase-level access characteristics to adjust locking scopes dynamically. This adaptive approach ensures both economic efficiency and service quality, addressing the unique availability constraints of agent-centric applications.

Hybrid Concurrency Control (Hybrid CC). Hybrid CC combines the strengths of OCC and PCC to handle diverse workloads. Composing CC [96] provides a framework for safely composing different CC protocols. Dynamic OCC [17] executes transactions optimistically, then re-executes aborted ones using 2PL based on the collected read/write set during the previous run. CormCC [71] partitions data based on historical access patterns and assigns each partition a fixed policy (OCC or PCC), dynamically adjusting boundaries to adapt to contention/hotspot shifts. MOCC [77] integrates ordered locking into PCC to reduce deadlocks. Plor [19] enhances wound-wait PCC with optimistic reads, reducing unnecessary lock blocking.

However, all of them face challenges in agentic workloads. Dynamic OCC assumes *deterministic re-execution*, locking the observed read/write set. This fails with non-deterministic LLM agents that may generate entirely different SQL statements upon retry, rendering captured locks useless. For CormCC, its coarse-grained, partition-level adaptation ignores the heterogeneity in agents' behaviors accessing the same data, resulting in mismatched strategies. MOCC and Plor lack semantic awareness. They treat token-expensive agent transactions the same as traditional transactions, often leading to the preemption of high-value tasks.

Priority-Based Concurrency Control. Priority-based concurrency control is a suitable scheme to prioritize certain classes of transactions over others, ensuring low tail latency of critical transactions. It intuitively seems suitable for prioritizing interactive transactions, aligning closely with our approach. Existing systems such as Microsoft SQL Server [2], Oracle Berkeley DB [4], and CockroachDB [69] incorporate ad hoc prioritization mechanisms to favor high-priority transactions in scheduling or validation. More structured academic approaches, such as Polaris [86] and Ding et al. [25], refine this by integrating priority into optimistic validation. Polaris uses a lightweight reservation mechanism based on timestamps and abort counts to guarantee progress for older, retry-heavy transactions, while Ding et al. defer conflict resolution to validate prioritized batches sequentially.

However, relying on static heuristics (e.g., timestamps or abort counts) proves insufficient for agentic workloads. In optimistic settings, over-prioritizing conflicting high-priority transactions triggers aborts among agentic transactions, while in pessimistic modes, aggressive lock preemption leads to starvation. Crucially, these static schemes lack semantic awareness of *transaction value*. They fail to account for the unique runtime characteristics of interactive agents, such as variable urgency, evolving access patterns, and token consumption. Aegis overcomes this by defining priority not as a static label, but as a dynamic function of real-time contention and accumulated token investment, ensuring that high-value agents survive without crippling system-wide throughput.

9 Conclusion

In this paper, we present Aegis, an adaptive concurrency control framework tailored for the emerging paradigm of agentic transactions. Aegis introduces a reinforcement learning-based policy to dynamically adapt execution strategies, switching between optimistic and pessimistic modes based on the runtime interpretation of agent behaviors. Transactions identified as high-risk or cost-sensitive are proactively protected via lock scheduling, while others proceed optimistically to maximize concurrency. By effectively balancing the trade-off between immediate blocking overhead and future abort

costs, Aegis addresses the unique challenges of long-running, non-deterministic agentic workflows. It preserves the high throughput advantage of traditional schemes while providing the robustness and low tail latency required for LLM-driven data agents.

References

[1] 2026. LangGraph. https://docs.langchain.com/oss/python/langgraph/overview.
[2] 2026. Microsoft SQL Server. https://www.microsoft.com/en-us/sql-server.
[3] 2026. openGauss-MOT. https://docs.opengauss.org/en/docs/latest/database_administration_guide/mot.html/.
[4] 2026. Oracle Berkeley DB. https://www.oracle.com/database/technologies/related/berkeleydb.html.
[5] 2026. Ozai. https://www.getburnt.ai/about.
[6] 2026. TPC-C. https://www.tpc.org/tpc_documents_current_versions/pdf/tpc-c_v5.11.0.pdf.
[7] 2026. Workday. https://www.workday.com/en-us/artificial-intelligence/ai-agents.html.
[8] 2026. YCSB. https://github.com/brianfrankcooper/YCSB/.
[9] Daniel J. Abadi, Samuel R. Madden, and Nabil Hachem. 2008. Column-stores vs. row-stores: how different are they really?. In *Proceedings of the 2008 ACM SIGMOD International Conference on Management of Data* (Vancouver, Canada) *(SIGMOD '08)*. Association for Computing Machinery, New York, NY, USA, 967–980. doi:10.1145/1376616.1376712
[10] Deepak Bhaskar Acharya, Karthigeyan Kuppan, and B Divya. 2025. Agentic AI: Autonomous intelligence for complex goals—A comprehensive survey. *IEEe Access* 13 (2025), 18912–18936.
[11] M. M. Astrahan, M. W. Blasgen, D. D. Chamberlin, K. P. Eswaran, J. N. Gray, P. P. Griffiths, W. F. King, R. A. Lorie, P. R. McJones, J. W. Mehl, G. R. Putzolu, I. L. Traiger, B. W. Wade, and V. Watson. 1976. System R: relational approach to database management. 1, 2 (June 1976), 97–137. doi:10.1145/320455.320457
[12] Hillel Avni, Alisher Aliev, Oren Amor, Aharon Avitzur, Ilan Bronshtein, Eli Ginot, Shay Goikhman, Eliezer Levy, Idan Levy, Fuyang Lu, et al. 2020. Industrial-strength OLTP using main memory and many cores. *Proceedings of the VLDB Endowment* 13, 12 (2020), 3099–3111.
[13] Peter Bailis, Alan Fekete, Michael J. Franklin, Ali Ghodsi, Joseph M. Hellerstein, and Ion Stoica. 2015. Feral Concurrency Control: An Empirical Investigation of Modern Application Integrity. In *Proceedings of the 2015 ACM SIGMOD International Conference on Management of Data* (Melbourne, Victoria, Australia) *(SIGMOD '15)*. Association for Computing Machinery, New York, NY, USA, 1327–1342. doi:10.1145/2723372.2737784
[14] Philip A Bernstein, Vassos Hadzilacos, Nathan Goodman, et al. 1987. *Concurrency control and recovery in database systems*. Vol. 370. Addison-wesley Reading.
[15] Asim Biswal, Liana Patel, Siddharth Jha, Amog Kamsetty, Shu Liu, Joseph E. Gonzalez, Carlos Guestrin, and Matei Zaharia. 2025. Text2SQL is Not Enough: Unifying AI and Databases with TAG. In *Conference on Innovative Data Systems Research (CIDR)*. https://vldb.org/cidrdb/papers/2025/p11-biswal.pdf
[16] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. 2020. Language models are few-shot learners. *Advances in neural information processing systems* 33 (2020), 1877–1901.
[17] Man Cao, Minjia Zhang, Aritra Sengupta, and Michael D. Bond. 2016. Drinking from both glasses: combining pessimistic and optimistic tracking of cross-thread dependences. In *Proceedings of the 21st ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming* (Barcelona, Spain) *(PPoPP '16)*. Association for Computing Machinery, New York, NY, USA, Article 20, 13 pages. doi:10.1145/2851141.2851143
[18] Edward Y Chang and Longling Geng. 2025. SagaLLM: Context Management, Validation, and Transaction Guarantees for Multi-Agent LLM Planning. *Proceedings of the VLDB Endowment* 18, 12 (2025), 4874–4886.
[19] Youmin Chen, Xiangyao Yu, Paraschos Koutris, Andrea C. Arpaci-Dusseau, Remzi H. Arpaci-Dusseau, and Jiwu Shu. 2022. Plor: General Transactions with Predictable, Low Tail Latency. In *Proceedings of the 2022 International Conference on Management of Data* (Philadelphia, PA, USA) *(SIGMOD '22)*. Association for Computing Machinery, New York, NY, USA, 19–33. doi:10.1145/3514221.3517879
[20] James C Corbett, Jeffrey Dean, Michael Epstein, Andrew Fikes, Christopher Frost, Jeffrey John Furman, Sanjay Ghemawat, Andrey Gubarev, Christopher Heiser, Peter Hochschild, et al. 2013. Spanner: Google's globally distributed database. *ACM Transactions on Computer Systems (TOCS)* 31, 3 (2013), 1–22.
[21] Natacha Crooks, Youer Pu, Lorenzo Alvisi, and Allen Clement. 2017. Seeing is Believing: A Client-Centric Specification of Database Isolation. In *Proceedings of the ACM Symposium on Principles of Distributed Computing*. 73–82. doi:10.1145/3087801.3087802
[22] Eman Daraghmi, Cheng-Pu Zhang, and Shyan-Ming Yuan. 2022. Enhancing saga pattern for distributed transactions within a microservices architecture. *Applied Sciences* 12, 12 (2022), 6242.
[23] Sudipto Das, Divyakant Agrawal, and Amr El Abbadi. 2013. Elastras: An elastic, scalable, and self-managing transactional database for the cloud. *ACM Transactions on Database Systems (TODS)* 38, 1 (2013), 1–45.
[24] Cristian Diaconu, Craig Freedman, Erik Ismert, Per-Ake Larson, Pravin Mittal, Ryan Stonecipher, Nitin Verma, and Mike Zwilling. 2013. Hekaton: SQL server's memory-optimized OLTP engine. In *Proceedings of the 2013 ACM SIGMOD International Conference on Management of Data* (New York, New York, USA) *(SIGMOD '13)*. Association for Computing Machinery, New York, NY, USA, 1243–1254. doi:10.1145/2463676.2463710
[25] Bailu Ding, Lucja Kot, and Johannes Gehrke. 2018. Improving optimistic concurrency control through transaction batching and operation reordering. *Proc. VLDB Endow.* 12, 2 (Oct. 2018), 169–182. doi:10.14778/3282495.3282502
[26] Jose M. Faleiro, Daniel J. Abadi, and Joseph M. Hellerstein. 2017. High performance transactions via early write visibility. *Proc. VLDB Endow.* 10, 5 (Jan. 2017), 613–624. doi:10.14778/3055540.3055553
[27] Alan Fekete, Dimitrios Liarokapis, Elizabeth O'Neil, Patrick O'Neil, and Dennis Shasha. 2005. Making snapshot isolation serializable. *ACM Transactions on Database Systems (TODS)* 30, 2 (2005), 492–528.
[28] Longling Geng and Edward Y Chang. 2025. ALAS: Transactional and Dynamic Multi-Agent LLM Planning. *arXiv preprint arXiv:2511.03094* (2025).
[29] Shabnam Ghasemirad, Si Liu, Christoph Sprenger, Luca Multazzu, and David A. Basin. 2025. VerIso: Verifiable Isolation Guarantees for Database Transactions. *Proceedings of the VLDB Endowment* 18, 5 (2025), 1362–1375. doi:10.14778/3718057.3718065
[30] Jim Gray. 1981. The Transaction Concept: Virtues and Limitations. In *Proceedings of the 7th International Conference on Very Large Data Bases*. 144–154.
[31] Ken Gu, Ruoxi Shang, Ruien Jiang, Keying Kuang, Richard-John Lin, Donghe Lyu, Yue Mao, Youran Pan, Teng Wu, Jiaqian Yu, Yikun Zhang, Tianmai M. Zhang, Lanyi Zhu, Mike A. Merrill, Jeffrey Heer, and Tim Althoff. 2024. BLADE: Benchmarking Language Model Agents for Data-Driven Science. In *Findings of the Association for Computational Linguistics: EMNLP 2024*. 13936–13971. https://aclanthology.org/2024.findings-emnlp.815/
[32] Long Gu, Si Liu, Tiancheng Xing, Hengfeng Wei, Yuxing Chen, and David Basin. 2024. IsoVista: Black-box Checking Database Isolation Guarantees. *Proceedings of the VLDB Endowment* 17, 12 (2024), 4325–4328. doi:10.14778/3685800.3685866
[33] Zhihan Guo, Kan Wu, Cong Yan, and Xiangyao Yu. 2021. Releasing Locks As Early As You Can: Reducing Contention of Hotspots by Violating Two-Phase Locking. In *Proceedings of the 2021 International Conference on Management of Data* (Virtual Event, China) *(SIGMOD '21)*. Association for Computing Machinery, New York, NY, USA, 658–670. doi:10.1145/3448016.3457294
[34] Theo Haerder and Andreas Reuter. 1983. Principles of Transaction-Oriented Database Recovery. *Comput. Surveys* 15, 4 (1983), 287–317. doi:10.1145/289.291
[35] Chaoyue He, Xin Zhou, Di Wang, Hong Xu, Wei Liu, and Chunyan Miao. 2026. Harness engineering for language agents: The harness layer as control, agency, and runtime. (2026).
[36] Wei He, Yueqing Sun, Hongyan Hao, Xueyuan Hao, Zhikang Xia, Qi Gu, Chengcheng Han, Dengchang Zhao, Hui Su, Kefeng Zhang, et al. 2025. Vitabench: Benchmarking llm agents with versatile interactive tasks in real-world applications. *arXiv preprint arXiv:2509.26490* (2025).
[37] Gansen Hu, Zhaoguo Wang, Chuzhe Tang, Jiahuan Shen, Zhiyuan Dong, Sheng Yao, and Haibo Chen. 2024. WeBridge: Synthesizing Stored Procedures for Large-Scale Real-World Web Applications. *Proc. ACM Manag. Data* 2, 1, Article 64 (March 2024), 29 pages. doi:10.1145/3639319
[38] Xueyu Hu, Ziyu Zhao, Shuang Wei, Ziwei Chai, Qianli Ma, Guoyin Wang, Xuwu Wang, Jing Su, Jingjing Xu, Ming Zhu, Yao Cheng, Jianbo Yuan, Jiwei Li, Kun Kuang, Yang Yang, Hongxia Yang, and Fei Wu. 2024. InfiAgent-DABench: Evaluating Agents on Data Analysis Tasks. In *Proceedings of the 41st International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 235)*. PMLR, 19544–19572. https://proceedings.mlr.press/v235/hu24s.html
[39] Liqiang Jing, Zhehui Huang, Xiaoyang Wang, Wenlin Yao, Wenhao Yu, Kaixin Ma, Hongming Zhang, Xinya Du, and Dong Yu. 2025. DSBench: How Far Are Data Science Agents from Becoming Data Science Experts?. In *International Conference on Learning Representations (ICLR)*. https://openreview.net/forum?id=DSsSPr0RZJ
[40] Kostis Kaffes, Eugene Wu, et al. 2026. Agentic Data Environments. *IEEE Data Engineering Bulletin* (2026). https://www.cs.columbia.edu/~kkaffes/papers/agenticdataenvs-ieee2026.pdf
[41] Fangyu Lei, Jixuan Chen, Yuxiao Ye, Ruisheng Cao, Dongchan Shin, Hongjin Su, Zhaoqing Suo, Hongcheng Gao, Wenjing Hu, Pengcheng Yin, Victor Zhong, Caiming Xiong, Ruoxi Sun, Qian Liu, Sida I. Wang, and Tao Yu. 2025. Spider 2.0: Evaluating Language Models on Real-World Enterprise Text-to-SQL Workflows. In *International Conference on Learning Representations (ICLR)*. https://openreview.net/forum?id=XmProj9cPs
[42] Guohao Li, Hasan Hammoud, Hani Itani, Dmitrii Khizbullin, and Bernard Ghanem. 2023. Camel: Communicative agents for" mind" exploration of large language model society. *Advances in neural information processing systems* 36 (2023), 51991–52008.

[43] Junjie Li, Xi Xiao, Yunbei Zhang, Chen Liu, Lin Zhao, Xiaoying Liao, Yingrui Ji, Janet Wang, Jianyang Gu, Yingqiang Ge, et al. 2026. Agent Harness Engineering: A Survey. *OpenReview preprint* (2026).

[44] Yang Li, Siqi Ping, Xiyu Chen, Xiaojian Qi, Zigan Wang, Ye Luo, and Xiaowei Zhang. 2025. AgentGit: A Version Control Framework for Reliable and Scalable LLM-Powered Multi-Agent Systems. *arXiv preprint arXiv:2511.00628* (2025).

[45] Hyeontaek Lim, Michael Kaminsky, and David G. Andersen. 2017. Cicada: Dependably Fast Multi-Core In-Memory Transactions. In *Proceedings of the 2017 ACM International Conference on Management of Data* (Chicago, Illinois, USA) *(SIGMOD '17)*. Association for Computing Machinery, New York, NY, USA, 21–35. doi:10.1145/3035918.3064015

[46] Xavier Limón, Alejandro Guerra-Hernández, Angel J. Sánchez-García, and Juan Carlos Pérez Arriaga. 2018. SagaMAS: A Software Framework for Distributed Transactions in the Microservice Architecture. In *2018 6th International Conference in Software Engineering Research and Innovation (CONISOFT)*. 50–58. doi:10.1109/CONISOFT.2018.8645853

[47] Jiahang Lin, Shichun Liu, Chengjun Pan, Lizhi Lin, Shihan Dou, Zhiheng Xi, Xuanjing Huang, Hang Yan, Zhenhua Han, Tao Gui, et al. 2026. Agentic harness engineering: Observability-driven automatic evolution of coding-agent harnesses. *arXiv preprint arXiv:2604.25850* (2026).

[48] Chunwei Liu, Matthew Russo, Michael J. Cafarella, Lei Cao, Peter Baile Chen, Zui Chen, Michael J. Franklin, Tim Kraska, Samuel Madden, Rana Shahout, and Gerardo Vitagliano. 2025. Palimpzest: Optimizing AI-Powered Analytics with Declarative Query Processing. In *Conference on Innovative Data Systems Research (CIDR)*. https://vldb.org/cidrdb/papers/2025/p12-liu.pdf

[49] Si Liu, Luca Multazzu, Hengfeng Wei, and David A. Basin. 2024. NOC-NOC: Towards Performance-Optimal Distributed Transactions. *Proceedings of the ACM on Management of Data* 2, 1, Article 9 (2024), 25 pages. doi:10.1145/3639264

[50] Shu Liu, Soujanya Ponnapalli, Shreya Shankar, Sepanta Zeighami, Alan Zhu, Shubham Agarwal, Ruiqi Chen, Samion Suwito, Shuo Yuan, Ion Stoica, Matei Zaharia, Alvin Cheung, Natacha Crooks, Joseph E. Gonzalez, and Aditya G. Parameswaran. 2026. Supporting Our AI Overlords: Redesigning Data Systems to be Agent-First. In *Conference on Innovative Data Systems Research (CIDR)*. https://www.vldb.org/cidrdb/papers/2026/p32-liu.pdf

[51] Yi Lu, Xiangyao Yu, and Samuel Madden. 2019. STAR: scaling transactions through asymmetric replication. *Proceedings of the VLDB Endowment* 12, 11 (jul 2019), 1316–1329. doi:10.14778/3342263.3342270

[52] Yuyu Luo, Guoliang Li, Ju Fan, and Nan Tang. 2026. Data Agents: Levels, State of the Art, and Open Problems. In *Companion of the International Conference on Management of Data* (India) *(SIGMOD Companion '26)*. Association for Computing Machinery, New York, NY, USA, 571–579. doi:10.1145/3788853.3801878

[53] Qianyu Meng, Yanan Wang, Liyi Chen, Qimeng Wang, Chengqiang Lu, Wei Wu, Yan Gao, Yi Wu, and Yao Hu. 2026. Agent harness for large language model agents: A survey. (2026).

[54] Bardia Mohammadi, Nearchos Potamitis, Lars Klein, Akhil Arora, and Laurent Bindschaedler. 2026. Atomix: Timely, Transactional Tool Use for Reliable Agentic Workflows. *arXiv preprint arXiv:2602.14849* (2026).

[55] C. Mohan, Don Haderle, Bruce Lindsay, Hamid Pirahesh, and Peter Schwarz. 1992. ARIES: A Transaction Recovery Method Supporting Fine-Granularity Locking and Partial Rollbacks Using Write-Ahead Logging. *ACM Transactions on Database Systems* 17, 1 (1992), 94–162. doi:10.1145/128765.128770

[56] Andrea Muttoni and Jason Zhao. 2025. Agent TCP/IP: An Agent-to-Agent Transaction System. *arXiv preprint arXiv:2501.06243* (2025).

[57] Cuong D. T. Nguyen, Kevin Chen, Christopher DeCarolis, and Daniel J. Abadi. 2025. Are Database System Researchers Making Correct Assumptions about Transaction Workloads? *Proc. ACM Manag. Data* 3, 3, Article 131 (June 2025), 26 pages. doi:10.1145/3725268

[58] Christos H. Papadimitriou. 1979. The Serializability of Concurrent Database Updates. *J. ACM* 26, 4 (1979), 631–653. doi:10.1145/322154.322158

[59] Liana Patel, Siddharth Jha, Melissa Z. Pan, Harshit Gupta, Parth Asawa, Carlos Guestrin, and Matei Zaharia. 2025. Semantic Operators and Their Optimization: Enabling LLM-Based Data Processing with Accuracy Guarantees in LOTUS. *Proceedings of the VLDB Endowment* 18, 11 (2025), 4171–4184. doi:10.14778/3749646.3749685

[60] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, Ilya Sutskever, et al. 2019. Language models are unsupervised multitask learners. *OpenAI blog* 1, 8 (2019), 9.

[61] Kun Ren, Alexander Thomson, and Daniel J Abadi. 2012. Lightweight locking for main memory database systems. *Proceedings of the VLDB Endowment* 6, 2 (2012), 145–156.

[62] Matthew Russo and Tim Kraska. 2026. Deep Research is the New Analytics System: Towards Building the Runtime for AI-Driven Analytics. In *Conference on Innovative Data Systems Research (CIDR)*. https://www.vldb.org/cidrdb/papers/2026/p30-russo.pdf

[63] Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. 2023. Toolformer: Language models can teach themselves to use tools. *Advances in Neural Information Processing Systems* 36 (2023), 68539–68551.

[64] Yangjun Sheng, Anthony Tomasic, Tieying Zhang, and Andrew Pavlo. 2019. Scheduling OLTP transactions via learned abort prediction. In *Proceedings of the Second International Workshop on Exploiting Artificial Intelligence Techniques for Data Management* (Amsterdam, Netherlands) *(aiDM '19)*. Association for Computing Machinery, New York, NY, USA, Article 1, 8 pages. doi:10.1145/3329859.3329871

[65] Michael Stonebraker and Uğur Çetintemel. 2018. " One size fits all" an idea whose time has come and gone. In *Making databases work: the pragmatic wisdom of Michael Stonebraker*. 441–462.

[66] Michael Stonebraker, Gerald Held, Eugene Wong, and Peter Kreps. 1976. The design and implementation of INGRES. *ACM Trans. Database Syst.* 1, 3 (Sept. 1976), 189–222. doi:10.1145/320473.320476

[67] Maojun Sun, Ruijian Han, Binyan Jiang, Houduo Qi, Defeng Sun, Yancheng Yuan, and Jian Huang. 2026. Lambda: A large model based data agent. *J. Amer. Statist. Assoc.* 121, 553 (2026), 1–13.

[68] Zhaoyan Sun, Jiayi Wang, Xinyang Zhao, Jiachi Wang, and Guoliang Li. 2025. Data agent: A holistic architecture for orchestrating data+ ai ecosystems. *arXiv preprint arXiv:2507.01599* (2025).

[69] Rebecca Taft, Irfan Sharif, Andrei Matei, Nathan VanBenschoten, Jordan Lewis, Tobias Grieger, Kai Niemi, Andy Woods, Anne Birzin, Raphael Poss, Paul Bardea, Amruta Ranade, Ben Darnell, Bram Gruneir, Justin Jaffray, Lucy Zhang, and Peter Mattis. 2020. CockroachDB: The Resilient Geo-Distributed SQL Database. In *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data* (Portland, OR, USA) *(SIGMOD '20)*. Association for Computing Machinery, New York, NY, USA, 1493–1509. doi:10.1145/3318464.3386134

[70] Chuzhe Tang, Zhaoguo Wang, Xiaodong Zhang, Qianmian Yu, Binyu Zang, Haibing Guan, and Haibo Chen. 2023. Ad Hoc Transactions: What They Are and Why We Should Care. *SIGMOD Rec.* 52, 1 (June 2023), 7–15. doi:10.1145/3604437.3604440

[71] Dixin Tang and Aaron J. Elmore. 2018. Toward Coordination-free and Reconfigurable Mixed Concurrency Control. In *2018 USENIX Annual Technical Conference (USENIX ATC 18)*. USENIX Association, Boston, MA, 809–822. https://www.usenix.org/conference/atc18/presentation/tang

[72] Kimi Team, Yifan Bai, Yiping Bao, Y Charles, Cheng Chen, Guanduo Chen, Haiting Chen, Huarong Chen, Jiahao Chen, Ningxin Chen, et al. 2025. Kimi k2: Open agentic intelligence. *arXiv preprint arXiv:2507.20534* (2025).

[73] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, et al. 2023. Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971* (2023).

[74] Stephen Tu, Wenting Zheng, Eddie Kohler, Barbara Liskov, and Samuel Madden. 2013. Speedy transactions in multicore in-memory databases. In *Proceedings of the Twenty-Fourth ACM Symposium on Operating Systems Principles* (Farminton, Pennsylvania) *(SOSP '13)*. Association for Computing Machinery, New York, NY, USA, 18–32. doi:10.1145/2517349.2522713

[75] Stephen Tu, Wenting Zheng, Eddie Kohler, Barbara Liskov, and Samuel Madden. 2013. Speedy transactions in multicore in-memory databases. In *Proceedings of the Twenty-Fourth ACM Symposium on Operating Systems Principles* (Farminton, Pennsylvania) *(SOSP '13)*. Association for Computing Machinery, New York, NY, USA, 18–32. doi:10.1145/2517349.2522713

[76] Jiachen Wang, Ding Ding, Huan Wang, Conrad Christensen, Zhaoguo Wang, Haibo Chen, and Jinyang Li. 2021. Polyjuice: High-Performance Transactions via Learned Concurrency Control. In *15th USENIX Symposium on Operating Systems Design and Implementation (OSDI 21)*. USENIX Association, 198–216. https://www.usenix.org/conference/osdi21/presentation/wang-jiachen

[77] Tianzheng Wang and Hideaki Kimura. 2016. Mostly-optimistic concurrency control for highly contended dynamic workloads on a thousand cores. *Proc. VLDB Endow.* 10, 2 (Oct. 2016), 49–60. doi:10.14778/3015274.3015276

[78] Todd Warszawski and Peter Bailis. 2017. ACIDRain: Concurrency-Related Attacks on Database-Backed Web Applications. In *Proceedings of the 2017 ACM International Conference on Management of Data* (Chicago, Illinois, USA) *(SIGMOD '17)*. Association for Computing Machinery, New York, NY, USA, 5–20. doi:10.1145/3035918.3064037

[79] Lianggui Weng, Dandan Liu, Rong Zhu, Bolin Ding, and Jingren Zhou. 2026. BridgeScope: A Universal Toolkit for Bridging Large Language Models and Databases. In *Conference on Innovative Data Systems Research (CIDR)*. https://www.vldb.org/cidrdb/papers/2026/p4-weng.pdf

[80] Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, et al. 2024. Autogen: Enabling next-gen LLM applications via multi-agent conversations. In *First conference on language modeling*.

[81] Hongyang Yang, Likun Lin, Yang She, Xinyu Liao, Jiaoyang Wang, Runjia Zhang, Yuquan Mo, and Christina Dan Wang. 2025. FinRobot: Generative Business Process AI Agents for Enterprise Resource Planning in Finance. *arXiv preprint arXiv:2506.01423* (2025).

[82] Xinjun Yang, Yingqiang Zhang, Hao Chen, Feifei Li, Bo Wang, Jing Fang, Chuan Sun, and Yuhui Wang. 2024. PolarDB-MP: A Multi-Primary Cloud-Native

Database via Disaggregated Shared Memory. In *Companion of the 2024 International Conference on Management of Data* (Santiago AA, Chile) *(SIGMOD-/PODS '24)*. Association for Computing Machinery, New York, NY, USA, 295–308. doi:10.1145/3626246.3653377

[83] Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik R Narasimhan. 2025. {τ}-bench: A Benchmark for \underline{T}ool-\underline{A}gent-\underline{U}ser Interaction in Real-World Domains. In *The Thirteenth International Conference on Learning Representations*. https://openreview.net/forum?id=roNSXZpUDN

[84] Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Tom Griffiths, Yuan Cao, and Karthik Narasimhan. 2023. Tree of thoughts: Deliberate problem solving with large language models. *Advances in neural information processing systems* 36 (2023), 11809–11822.

[85] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik R Narasimhan, and Yuan Cao. 2023. ReAct: Synergizing Reasoning and Acting in Language Models. In *The Eleventh International Conference on Learning Representations*. https://openreview.net/forum?id=WE_vluYUL-X

[86] Chenhao Ye, Wuh-Chwen Hwang, Keren Chen, and Xiangyao Yu. 2023. Polaris: Enabling Transaction Priority in Optimistic Concurrency Control. *Proc. ACM Manag. Data* 1, 1, Article 44 (May 2023), 24 pages. doi:10.1145/3588724

[87] Xiangyao Yu, Andrew Pavlo, Daniel Sanchez, and Srinivas Devadas. 2016. TicToc: Time Traveling Optimistic Concurrency Control. In *Proceedings of the 2016 International Conference on Management of Data* (San Francisco, California, USA) *(SIGMOD '16)*. Association for Computing Machinery, New York, NY, USA, 1629–1642. doi:10.1145/2882903.2882935

[88] Xiangyao Yu, Yu Xia, Andrew Pavlo, Daniel Sanchez, Larry Rudolph, and Srinivas Devadas. 2018. Sundial: harmonizing concurrency control and caching in a distributed oltp database management system. *Proceedings of the VLDB Endowment* 11, 10 (2018), 1289–1302.

[89] Aohan Zeng, Xin Lv, Qinkai Zheng, Zhenyu Hou, Bin Chen, Chengxing Xie, Cunxiang Wang, Da Yin, Hao Zeng, Jiajie Zhang, et al. 2025. Glm-4.5: Agentic, reasoning, and coding (arc) foundation models. *arXiv preprint arXiv:2508.06471* (2025).

[90] Ming Zhang, Yu Hua, Pengfei Zuo, and Lurong Liu. 2022. {FORD}: Fast One-sided {RDMA-based} Distributed Transactions for Disaggregated Persistent Memory. In *20th USENIX Conference on File and Storage Technologies (FAST 22)*. 51–68.

[91] Wayne Xin Zhao, Kun Zhou, Junyi Li, Tianyi Tang, Xiaolei Wang, Yupeng Hou, Yingqian Min, Beichen Zhang, Junjie Zhang, Zican Dong, et al. 2023. A survey of large language models. *arXiv preprint arXiv:2303.18223* 1, 2 (2023).

[92] Weixing Zhou, Qi Peng, Zijie Zhang, Yanfeng Zhang, Yang Ren, Sihao Li, Guo Fu, Yulong Cui, Qiang Li, Caiyi Wu, et al. 2023. GeoGauss: Strongly Consistent and Light-Coordinated OLTP for Geo-Replicated SQL Database. *Proceedings of the ACM on Management of Data* 1, 1 (2023), 1–27.

[93] Weixing Zhou, Yanfeng Zhang, Xinji Zhou, Zhiyou Wang, Zeshun Peng, Yang Ren, Sihao Li, Huanchen Zhang, Guoliang Li, and Ge Yu. 2025. Concurrency Control as a Service. *Proc. VLDB Endow.* 18, 9 (Sept. 2025), 2761–2774. doi:10.14778/3746405.3746406

[94] Yizhang Zhu, Liangwei Wang, Chenyu Yang, Xiaotian Lin, Boyan Li, Wei Zhou, Xinyu Liu, Zhangyang Peng, Tianqi Luo, Yu Li, et al. 2025. A Survey of Data Agents: Emerging Paradigm or Overstated Hype? *arXiv preprint arXiv:2510.23587* (2025).

[95] Qiyu Zhuang, Wei Lu, Shuang Liu, Yuxing Chen, Xinyue Shi, Zhanhao Zhao, Yipeng Sun, Anqun Pan, and Xiaoyong Du. 2025. TxnSails: Achieving Serializable Transaction Scheduling with Self-Adaptive Isolation Level Selection. *Proceedings of the VLDB Endowment* 18, 11 (2025), 4227–4240. doi:10.14778/3749646.3749689

[96] Ofri Ziv, Alex Aiken, Guy Golan-Gueta, G. Ramalingam, and Mooly Sagiv. 2015. Composing concurrency control. *SIGPLAN Not.* 50, 6 (June 2015), 240–249. doi:10.1145/2813885.2737970

A Proof of Correctness

This appendix provides the full proof of Theorem 5.5. We assume that tuple metadata updates on lock state, version, and committing status are atomic and immediately visible to concurrent threads.

We define the serialization point of a committed transaction T as the instant at which T successfully completes unified commit validation and obtains its commit version in Alg. 2. Since commit versions are unique and totally ordered, serialization points induce a total order over committed transactions.

Lemma 1 (Write conflicts are ordered by WLock). *For any two transactions T_i and T_j that both write the same tuple x, at most one of them can hold the write lock on x at a time. Therefore, their conflicting writes are ordered before both transactions can commit.*

Proof. Every transaction must invoke WLock on each tuple in its write set before entering the final commit phase. Since WLock grants exclusive ownership of the target tuple, two transactions cannot simultaneously hold the write lock on the same tuple. Thus, conflicting writers must be serialized through lock acquisition order, or one of them must wait or abort. □

Lemma 2 (Read–write conflicts are either blocked by locking or detected by validation). *Let T_r read tuple x, and let T_w be a concurrent transaction that writes x. If both transactions commit, then their conflict is ordered either by lock-based synchronization or by successful validation of the version observed by T_r.*

Proof. If T_r reads x under RLock, then T_w must observe the conflicting reader before obtaining WLock, and the conflict is explicitly ordered by the lock manager. If T_r reads x optimistically, it records the observed version and validates that version before commit. If T_w commits a write to x first, the version changes and T_r's validation fails. Hence, T_r can commit only if the observed version remains valid up to its serialization point. □

Lemma 3 (Deferred writes do not expose uncommitted or partially installed updates). *No transaction in* `Aegis` *can observe a buffered, uncommitted, or partially installed write of another transaction.*

Proof. All writes are buffered locally and are installed into shared tuples only during the commit phase, after successful lock acquisition and validation. Before physical installation, the transaction marks its write-locked tuples as committing, preventing optimistic readers from observing in-progress updates. Since SetCommitting is invoked only immediately before installation, readers may safely observe the old committed version before that point. Therefore, a concurrent transaction can observe only a previously committed version, a fully installed new version, or a retry condition, but never an uncommitted or partial update. □

Lemma 4 (Dynamic action changes preserve correctness). *If a transaction changes an access on tuple x from optimistic to lock-based execution, then retroactive validation ensures that the transaction cannot commit based on a stale earlier observation of x.*

Proof. When such an action change occurs, `Aegis` revalidates that the version of x still matches the version observed during the earlier optimistic access. If the version has changed, the transaction aborts. Therefore, a transaction cannot combine a stale optimistic observation with a later lock-based access to the same logical state. □

Lemma 5 (Priority affects scheduling but not correctness). *Priority-aware lock admission changes only the order in which waiting transactions obtain locks; it does not change the safety conditions for commit.*

Proof. A higher-priority transaction may be admitted before a lower-priority one only through the lock manager. However, every transaction must still acquire the required locks, validate optimistic reads, and pass the same commit protocol before installing writes.

Hence, priority changes scheduling order but does not permit a transaction to bypass conflict checks or expose uncommitted state. □

Theorem 1 (Conflict serializability of Aegis). *Every execution of committed transactions produced by* Aegis *is conflict-serializable.*

Proof. Consider the total order of committed transactions induced by their serialization points. By Lemma 1, all write–write conflicts are ordered by exclusive WLock. By Lemma 2, every read–write conflict is either ordered by locking or detected by validation before commit. By Lemma 3, writes become visible only after successful validation and are never exposed in a partial state. By Lemma 4, dynamic action changes cannot introduce stale dependencies that escape validation. Finally, by Lemma 5, priority-aware scheduling affects only progress order, not correctness conditions.

Therefore, every conflict edge between committed transactions is consistent with the total order of serialization points. The precedence graph over committed transactions is thus acyclic, which implies conflict serializability. □

B Agentic Workload

An Example: For a travel request (e.g., book SFO→JFK next Monday, prefer Star Alliance, avoid redeyes, budget < $600), the agent parses the request and generates an initial plan. Then, it queries the "Routes" table to retrieve flight segments and feasible itineraries. Based on intermediate results, it synthesizes additional SQL queries dynamically to address user constraints, such as filtering by alliance, time windows, or budget by querying "Routes," "Aircraft," and "Prices" tables. The agent iteratively optimizes candidate routes, relaxing constraints or exploring alternative joins if needed. Once feasible options are identified, it queries the "Seats" table to verify availability, selects a seat, and executes a write operation to finalize the reservation.